

Multimodal SLAM for Legged Robots in Unconstrained Environments

Rahul Aggarwal
School of Computer Science
Georgia Institute of Technology
Atlanta, USA
raggarwal43@gatech.edu

Abstract—We present a multimodal SLAM algorithm for quadrupeds that rely on factor graphs. Despite significant research into SLAM, there has yet to be a fully modular system with multiple sensor modalities that remains robust through numerous degenerate situations and is compatible with future experimentation without breaking. This motivates our proposed solution, which solves the problem of SLAM for legged robots by tightly fusing information from an IMU, lidar, and a camera, along with robot joint state information to jointly estimate the robot state and environment with minimal drift. Included in the design are deep-learned loop closures and semantic object tracking, all which help obtain a rich representation of a robot’s state and environment that is especially useful for downstream tasks. Our system has been benchmarked on simulated data, and while it does not show state of the art results, it is robust enough to use in many robotics scenarios.

Index Terms—Robotics, SLAM, quadruped

I. INTRODUCTION

Quadrupeds have come to the forefront of industry robotics applications, having an agility and ease of maneuverability over difficult terrain that traditional wheeled robots have significant trouble with. Their versatility makes them desirable not only in academic research but in various applications such as in construction sites, hazard inspection, and urban maintenance. However, with this potential versatility, in order to effectively perform autonomously for long periods of time, robots like this need a form of perception that enables them to localize themselves with respect to potentially unknown environments, ideally robust to a large variety in external situations. Indeed, the desired algorithm should then be able to leverage its accurate localization to develop a useful representation of its environment for downstream tasks.

To approach this problem, simultaneous localization and mapping, or SLAM was developed, with the goal of solving the previously ill-posed problem of solving for both an unknown state and environment at the same time. More specifically, SLAM seeks to regenerate the sequence of robot states along with a representation of the external map, often in occupancy or point cloud form, in order to perform better on navigation or task-specific control.

However, despite the significant amount of previous research into SLAM with legged robots, both with traditional state estimation [1] and multimodal SLAM [2], there has yet to be a fully modular system with capability of continued



Fig. 1. The custom Slambox hardware solution mounted upon a MiniCheetah. Pictured are the lidar, IMU, computer, camera, and trunk of the quadruped.

nominal usage even when one or more sensor modalities become degenerate. Typical degenerate situations with lidar sensors include self-similar environments, choke points, and large open areas, as each of these facilitate a specific failure case for which lidar only systems cannot recover a correct relative transformation and therefore suffer often irrecoverable drift. In the same vein, with visual only systems, while they can better localize themselves with semantic information, they fundamentally lose the robustness that 3D data guarantees. Even with proprioceptive methods, including legged kinematics and inertial data, only in ideal conditions does the drift suffer by a limited (yet constant) amount that an ideal algorithm would like to avoid. Our proposed solution aims to be robust to all such degenerate situations by combining sensor measurements in such a way that no single sensor can be a significant detriment to the accuracy of the system as a

whole.

A. Contributions

This paper makes the following contributions:

- Support for a wide variety of sensor modalities, with tightly coupled residuals, but loosely coupled data flow.
- A complete well-documented ROS package already tuned for use with legged robots, aimed at compatibility with many different sources of data.

II. PROBLEM STATEMENT

We would like to estimate both the history of poses and velocities of a legged robot that potentially has many sensors, with aims of being compatible with the software used in the ROS framework.

A. Frame Definitions

Our reference frames closely follow those mentioned in the ROS standard. Specifically, we have the map frame M , the odometry frame O , the base frame B , the IMU frame I , the camera frame S , and contact frame C and a lidar frame L .

B. State Definitions

Our robot state can be defined as follows:

$$X_i \stackrel{\text{def}}{=} [\mathbf{R}_i \in SO(3), \vec{p}_i \in \mathbb{R}^3, \vec{v}_i \in \mathbb{R}^3, b^g \in \mathbb{R}^3, b^a \in \mathbb{R}^3] \quad (1)$$

where R denotes the robot orientation, $\vec{p}$ and $\vec{v}$ denote the robot position and velocity respectively, and b^g, b^a denote the IMU gyroscope and accelerometer biases, respectively. In addition to the usual robot state, we additionally estimate a number of rotation invariant contact frames:

$$C \stackrel{\text{def}}{=} [\vec{c}_i \in \mathbb{R}^3] \quad (2)$$

where $\vec{c}_i$ denotes the position of an in-contact quadruped end-effector at any time i . We note that these might not be described at every time step, but rather some subset of the robot's state frames, subject to the speed and gait of the legged robot. Finally, we additionally estimate the position of landmarks obtained from semantic-depth information:

$$S \stackrel{\text{def}}{=} [\{\vec{s}_\ell \in \mathbb{R}^3 : \ell \in S_i\}] \quad (3)$$

where S_i denotes the index set for semantic landmarks at time step i . Landmarks may be shared across many time steps.

C. Measurement Definitions

Our measurements are defined as follows in accordance with notation from [2]:

$$\mathcal{Z} \stackrel{\text{def}}{=} \{\mathcal{I}_{ij}, \mathcal{F}_i, \mathcal{L}_i, \mathcal{C}_i\} \quad (4)$$

where $\mathcal{I}_{ij}$ denotes the set of IMU measurements received between time steps i and j , $\mathcal{F}_i$ denotes the joint angles $\tilde{\alpha}_i$ of each leg of the quadruped at time step i , $\mathcal{L}_i$ denotes a lidar measurement at time step i , and $\mathcal{C}_i$ denotes a camera/depth measurement at time step i .

We omit the details, but note that our optimization problem is a maximum-a-posteriori estimation of the likelihood of

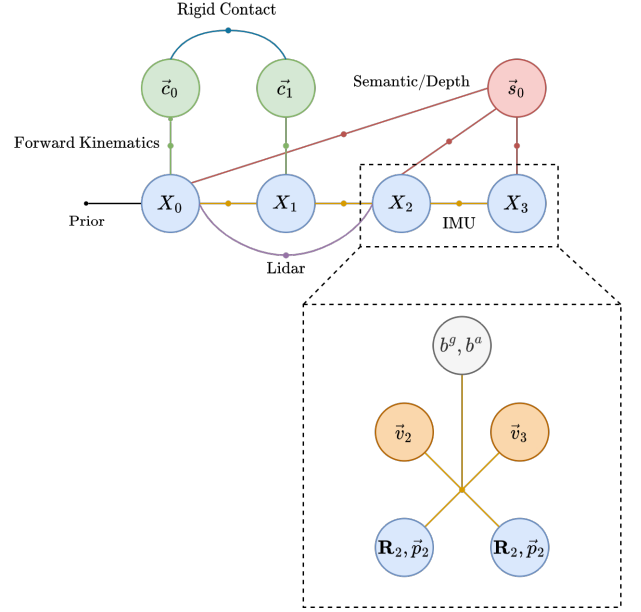


Fig. 2. Factor graph structure of our proposed algorithm. The factors include: prior (black), semantic/depth (red), lidar (purple) forward kinematics (green), rigid contact (blue), and inertial (orange). Not pictured are loop closure constraints.

measurements $\mathcal{Z}$ given the states X, C, S , which when solved, is a sum of squared residuals, inversely weighted by their covariance matrices (i.e the Mahalanobis Distance.).

III. FACTOR GRAPH FORMULATION

Here, we describe the factors that form the factor graph we optimize over to obtain the optimal state and any external features.

A. Inertial Factors

IMU preintegration theory has been discussed in depth in [3], so we omit the details here. Instead we note that IMU preintegration is a very effective of constraining the orientation, position, and velocity between two consecutive time steps within the graph without maintaining a set point of linearization. In this way, we can maintain an integration of high frequency signals from the IMU both to accumulate a high frequency state update (with drift) but also to jointly optimize with the rest of the sensor modalities upon a lower-frequency sensor data acquisition. For convenience, we denote the associated residual as $r_{\mathcal{I}_{ij}}$. We note that our proposed algorithm encodes a single pair of bias parameters b^a, b^g for the entire sequence of robot states; in practice, this causes the parameters to converge quickly and remain constant disregarding any sensor malfunction.

B. Lidar Factors

In contrast to previous works, we do not employ lidar feature tracking, opting for the much simpler and robust GICP (Generalized Iterative Closest Point) approach proposed by [4]. GICP tends to produce superior results by probabilistically

incorporating information from both traditional ICP and more accurate and robust point-to-plane approaches. Indeed, by tuning a hyperparameter c that represents the maximum threshold distance used to find corresponding point-point or point-plane pairs, GICP can be enough to guarantee robustness depending on the sensor. Lidar factors are introduced every time a sensor measurement comes in and are introduced into the factor graph as a relative pose factor between time steps i and keyframe time step k whose residual is as follows:

$$\vec{r}_{\mathcal{L}_i} = \text{Log}(\tilde{\mathbf{T}}_{ki}^{-1} \mathbf{T}_k^{-1} \mathbf{T}_i) \quad (5)$$

where $\tilde{\mathbf{T}}_{ki}$ is the estimated relative pose, $\mathbf{T}$ is the estimated pose of the lidar frame L with respect to the map frame L . Note that Log is an operator that takes elements from the $SE(3)$ manifold to their tangent space, where they can be optimized with traditional nonlinear techniques. The covariance here is assumed to be an isotropic gaussian, which we empirically determined to be a solid assumption. However, degenerate situations are resolved by incorporating robust cost functions [5] to limit the error introduced into the full optimization.

C. Legged Factors

Our proposed solution incorporates simplified hybrid contact factors first discussed in [6]. To that effect, we simplify the forward kinematic residual by noting that for many quadrupeds, the foot's (end effector) orientation remains constant with respect to the ground plane when in contact, and is therefore better modeled as a contact point rather than a contact frame or pose. Thus, our residual is as follows:

$$\vec{r}_{\mathcal{F}_i} = \mathbf{R}_i f(\tilde{\alpha}_i) + \vec{p}_i - \vec{c}_i \quad (6)$$

where f represents the forward kinematic function of an arbitrary leg of the quadruped. Notably, we do not use the body manipulator Jacobian described in [6] to define the covariance associated with the residual, instead opting to define $\Sigma_{\mathcal{F}}$ as an isotropic gaussian with a robust cost function. In a similar vein, because we reduce our contact factors from poses in $SE(3)$ to just vectors in $\mathbb{R}^3$, our hybrid rigid contact residual changes as the following:

$$\vec{r}_{\mathcal{C}_{ij}} = \vec{c}_j - \vec{c}_i - \Delta \tilde{c}_{ij} \quad (7)$$

where $\Delta \tilde{c}_{ij}$ represents the measured change in contact point across the hybrid switching surface or through stationary stances. The associated covariance changes in dimension but remains a matrix that accumulates higher variances across more time and switches.

D. Loop Closure Factors

Loop closures are done using visual information from the camera. Specifically, our proposed solution passes every n th image frame through a Resnet [7] backbone and uses the corresponding 2048-dimensional latent space as a search space for candidate loop closures. Once candidates have been found, we take the two lidar point clouds used for loop closure

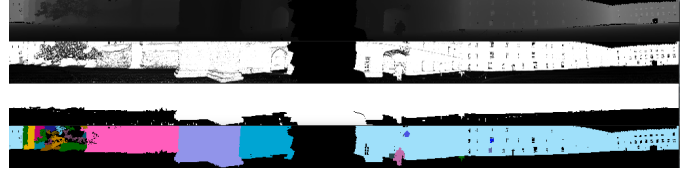


Fig. 3. The results of the segmentation pipeline. *Top*: A grayscale view from the camera. *Top-middle*: The camera view with inaccurate depths filtered. *Bottom-middle*: The camera view with ground pixels filtered out. *Bottom*: The camera view with pixels segmented according to instance and category.

verification and formulate a residual similar to the previously discussed lidar factors:

$$\vec{r}_{\mathcal{O}_i} = \text{Log}(\tilde{\mathbf{T}}_{ki}^{-1} \mathbf{T}_k^{-1} \mathbf{T}_i) \quad (8)$$

The only fundamental difference between a loop closure factor and a lidar factor is the covariance matrix $\Sigma_{\mathcal{O}}$, which we map to be the confidence of the associated loop closure. We measure this confidence to be a function of the distance within the latent space and the corresponding geometric distance.

E. Semantic/Depth Factors

There is one main factor related to semantic (camera-based) and depth measurements. As in [8], we pass the RGB images from the camera into a semantic/instance segmentation network, taking the resulting bounding boxes and category labels and aligning them into the corresponding depth images. Note that this process requires a calibrated stereo/camera/depth setup in order to achieve accurate results. Given the calibrated camera parameters and the depth image, we back project pixels belonging to each instance into 3D coordinates in the base frame B using the following equation:

$$\vec{s}_i^{jk} = \mathbf{D}_i(\vec{u}_i^{jk}) \mathbf{K}^{-1} \vec{u}_i^{jk} \quad (9)$$

where $\vec{u}_i^{jk} = (u, v, 1)^T$ represents the j th pixel of the k th instance in a camera image at time step i . The map $\mathbf{D}_i$ provides calibrated measurements at each image pixel $\vec{u}$ and the resulting backprojection $\vec{s}_i^{jk} = (x, y, z)^T$ represents the j th 3d point of the k instance at time step i . As noted before, each point $\vec{s}$ is given relative to the base frame of the robot B . To track segmented instances across frames, we employ a combination of the latent vector space of the segmentation network as well as a loose geometric constraint to provide matching candidate. Once we have identified it as a landmark we have discovered before, say, $\vec{s}_i$, we take the centroid of each instance $\vec{s}_{i,\ell}^k = \frac{1}{J} \sum_j \vec{s}_{i,\ell}^{jk}$ and add it to the factor graph. The corresponding residual is as follows:

$$\vec{r}_{X_i, \vec{s}_\ell} = \mathbf{R}_i \vec{s}_\ell + \vec{p}_i - \vec{s}_i \quad (10)$$

which is implemented as a simple pose to point factor. Because we do not maintain strong probabilistic data association in the frontend, we maintain larger covariances for these types of factor and robust cost functions for inaccurate registrations.

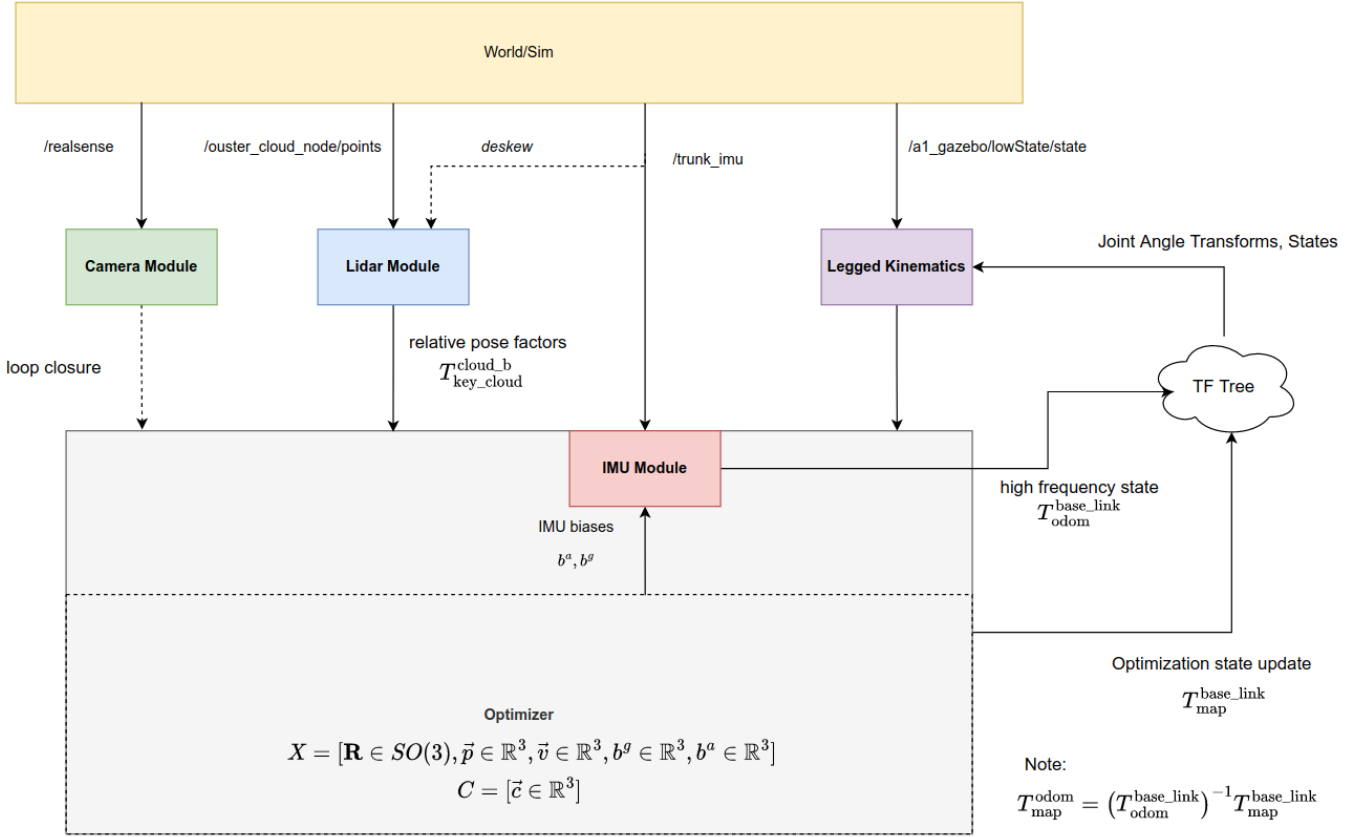


Fig. 4. Block diagram of our proposed solution. Each separate module contains a process that runs completely independent of one another, communicating through the ROS TCP protocols. Pictured are topic names, TF-tree updates, and data flow.

IV. IMPLEMENTATION

The architecture for our proposed solution is displayed in Fig. 4. One key contribution of our system is the complete modularity of each sensor modality. In this case, we take advantage of the ROS TCP network stack to isolate each sensor into their own process, keeping data transfer across a minimal surface. Importantly, the IMU propagates a high frequency but constant drift state that acts as the transform between the base frame B of the robot and the odometry frame O as specified by the ROS standard. After each optimization pass is complete, the system will then publish a transform from the odometry frame to the map frame M, which can then be used to recover the state of the robot in the map frame itself. The factor graph is solved using an open source solution called GTSAM [9], particularly the optimizer iSAM2 [10], which provides a consistent, robust, accurate, and performant backbone for our proposed solution.

A. Time Step Synchronization

In order to properly constrain the many factors from the multimodal suite of sensors we include in our system, we must properly synchronize the various sensor data. With a factor graph implementation, there is a contrasting level of performance from either a sparse or dense graph. In other words,

at every time step t , any number of sensors may have given a measurement slightly before or after a specific linearization point. However, in order to have a fully constrained system, we must group this sensor information in a consistent manner. Specifically, we employ a post-synchronization scheme, where both the legged kinematics, lidar, and loop closure information can provide a new state X_t . Because each sensor is processed separately and subsequent information is transmitted through the ROS TCP network configuration, drift in synchronization and unordered packets may arrive. Therefore, when processing a new piece of sensor data, our algorithm searches through a fixed circular buffer of previous time step states to find a close state within a certain margin, say δ , which we empirically determined to be 1 ms for best performance. This synchronization ensures not only that a minimum number of nodes gets added to the graph, preserving computational time, but also maintains a tight coupling between the sensor modalities, keeping the optimization problem relatively well constrained. In [11], lidar data that was captured during motion was compensated for using the high-frequency IMU propagation; however, we have observed no significant error resulting from inter-motion lidar captures that contributes significantly to drift.

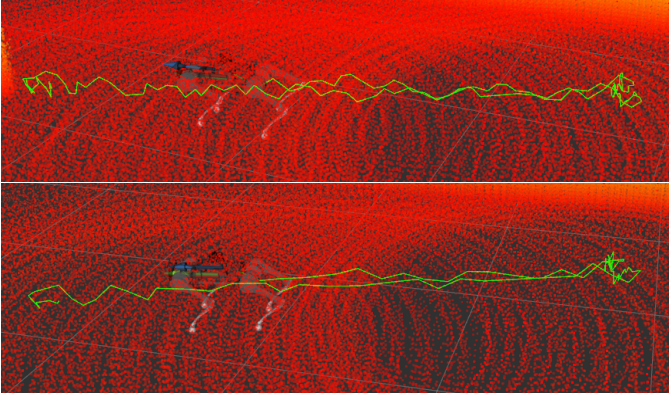


Fig. 5. The effect of varying the keyframe distance hyperparameter ϵ . *Above*: A keyframe distance of 0.05m, which exhibits characteristic jitteriness resulting from the motion of the quadruped kinematics. *Below*: A keyframe distance of 0.1m, which smooths out the trajectory but loses detail in the sinusoidal motion of the quadruped trunk.

B. Lidar Keyframes

As shown in [12] lidar keyframes provide much more accurate pose estimation through ICP over longer horizons. To be specific, we introduce a hyperparameter ϵ representing the distance the quadruped can travel before a new lidar keyframe is introduced. Once the new lidar keyframe is introduced, all subsequent lidar measurements before the next will be registered and integrated into the current lidar keyframe as a reference cloud, therefore allowing for higher quality and more robust results. The result of varying ϵ can be seen in Fig. 5. Along with higher robustness, lidar keyframes significantly reduce the memory requirements of our proposed system by limiting the number of dense point clouds stored at any one time.

C. Calibration

For semantic information, we need both external calibration as well as camera calibration. The IMU, lidar, camera and base positions are all rigidly collocated and are obtained from a custom CAD model called the Slambox, depicted in Fig. 1 which we mount upon the quadruped. Time synchronization is not performed due to hardware constraints, but instead compensated for in our proposed system as mentioned in section IV-A.

D. IMU Initialization

Because of our choice to hold a singular pair of biases b^a, b^g throughout the optimization procedure, our resulting pose-graph is not robust to poor IMU initialization as can be seen in Fig. 6. Notably, in order to compensate for this, we can run the system on given data for a small amount of time, recover the converged biases, and then restart the system with the proper biases to achieve correct initialization. For real-time performance, the sparsity and long-horizon nature of factor graphs ensure that error from initial poses do not contribute significantly to future error except in the case of loop closures.

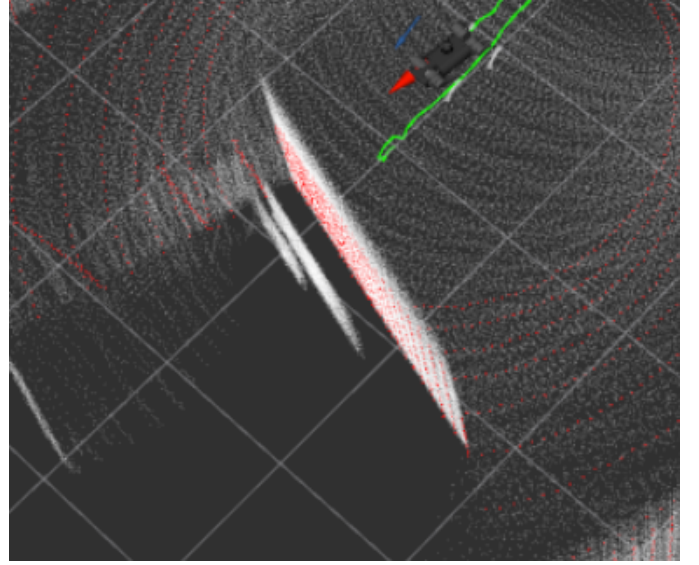


Fig. 6. The affect of subpar IMU initialization on the generated map. There are clear artifacts generated from the combination of the lidar and the IMU constraints as they hallucinate the wall to an error of ~ 0.3 m.

E. Spurious Contact Noise

Contact noise is a large problem in hybrid systems as the switching surface may experience spurious interrupts that result in an under-constrained system. To combat this problem, we introduce a small delay $\gamma > 0$ such that when a contact switch event occurs, no future contact switch events may occur on that same leg for γ seconds, ensuring we keep a constrained optimization problem.

V. EXPERIMENTAL RESULTS

A. Experimental Setup

To collect our data, we collected multiple trajectories in simulation that varied initial orientations, translational and rotational velocities, trajectories, and room locations in order to get a plethora of data to test our proposed algorithm. In particular, our sensor measurements emulated the Ouster OS0-32 lidar sensor, the Intel Realsense D435i depth camera, the Vectornav VN-100 IMU, and the Unitree Robotics joint state messages. Ground truth was obtained from the Gazebo simulation that the trajectories were obtained in.

Table I provides specific information on each recorded trajectory and the associated characteristics. Average rotational velocity was omitted for brevity and due to the fact that many trajectories employed a "move forward" then "rotate" philosophy. Our proposed solution was evaluated only against the ground truth provided by the Gazebo simulator due to time constraints, and we chose to use absolute pose error (APE) as our metric to evaluate performance. To do this, a separate ROS node would receive the ground truth and sample the latest pose in the optimized trajectory to compare and generate results while accounting for different frequencies of output. Finally, we tested our system on a computer with an Intel CoreTM i7-8750H CPU @ 2.20GHz with an NVIDIA GeForce RTX

TABLE I
TRAJECTORIES

#	Duration (s)	Total Distance (m)	Avg. Velocity (m/s)
1	22	30	1.36
2	67	36	0.53
3	93	158	1.69
4	15	8	0.53
5	49	38	0.77
6	18	12	0.67

TABLE II
RMSE ABSOLUTE POSE ERROR

#	APE	APE Translation (m)	APE Rotation (deg)
1	0.044	0.023	0.072
2	0.12	0.006	0.11
3	1.18	0.89	0.42
4	1.34	0.22	1.18
5	0.56	0.11	0.43
6	0.23	0.08	0.89

2070-Mobile GPU. Our system runs in real-time, so we omit timing from the results.

B. Results

Quantitative results are summarized in Table II. It is clear from the results that our proposed solution performs significantly better on trajectories with low amounts of rotation compared to translation, as can be seen by the higher-on-average numbers for the absolute pose error in rotation compared to translation. Figure 7 visualizes some of the aforementioned trajectories, and once can see that the obtained maps fit nicely with the ground truth, up to visual artifacts. In all cases, the resulting errors were well within the scope of error for an implementation of this size, and shows that our proposed solution generates good results on a variety of simulated trajectories. Indeed, we chose not to vary any hyperparameters for these tests, instead defaulting to those already empirically determined, which demonstrates that our proposed solution may be used as a robust upstream localizer for downstream tasks.

VI. CONCLUSION

We present a multimodal SLAM algorithm in a complete and robust ROS package made specifically for legged robots, namely quadrupeds. We employed the use of commonly used factor graph optimizers with a wide variety of sensors specifically in order to combat degenerate scenarios for each of these sensors. By tightly coupling each of the aforementioned residuals and employing robust cost functions, we maintain a low-drift system that remains in steady-state when sensors are either not available by configuration of the robot or through faulty sensor input. We presented a full analysis of the many smaller issues present within the system as well as our proposed workarounds, and concluded by benchmarking our system in simulation.

We note that the purpose of our solution is not to beat the state of the art but rather to provide a robust upstream

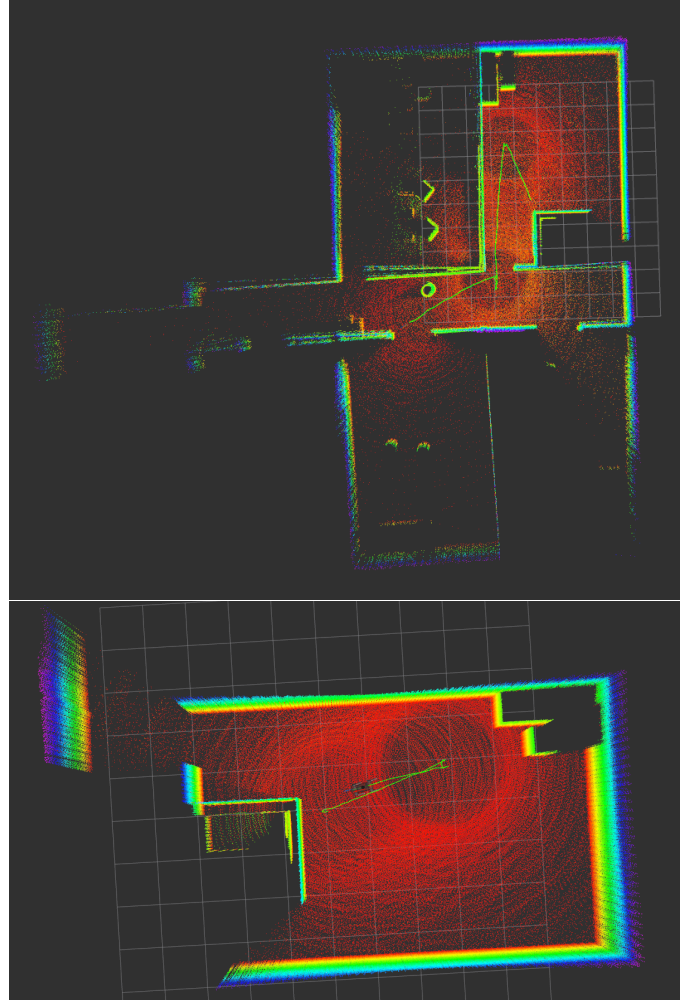


Fig. 7. *Above:* The robots estimated optimal trajectory through simulated trajectory #3. The map is extremely confident within its starting room and begins to gain confidence after it walks through a doorway, which is a common degenerate scenario. *Below:* The robot's estimated optimal trajectory through simulated trajectory #5. The only artifacts here are from poor IMU initialization. Both visualizations present the poses as they are optimized, instead of replaying the fully optimized trajectory.

application for downstream tasks such as navigation or exploration, which require robust, consistent state estimation with minimal hyperparameter tuning. Our solution is written completely in C++ and exhibits real-time performance with modern hardware typically found on quadrupeds.

Further work may involve more extensive benchmarking with real-world data and against other state of the art algorithms, including odometry methods included on many modern quadrupeds. We note that the modularity of our system allows future extension with new state estimation techniques such as neural scene representations.

REFERENCES

- [1] M. Bloesch, M. Burri, H. Sommer, R. Siegwart, and M. Hutter, "The two-state implicit filter recursive estimation for mobile robots," *IEEE Robotics and Automation Letters*, vol. 3, no. 1, pp. 573–580, 2017.

- [2] D. Wisth, M. Camurri, and M. Fallon, "Vilens: Visual, inertial, lidar, and leg odometry for all-terrain legged robots," *IEEE Transactions on Robotics*, 2022.
- [3] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, "On-manifold preintegration for real-time visual-inertial odometry," *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, 2016.
- [4] K. Koide, M. Yokozuka, S. Oishi, and A. Banno, "Voxelized gicp for fast and accurate 3d point cloud registration," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 11 054–11 059.
- [5] K. MacTavish and T. D. Barfoot, "At all costs: A comparison of robust cost functions for camera correspondence outliers," in *2015 12th conference on computer and robot vision*. IEEE, 2015, pp. 62–69.
- [6] R. Hartley, M. G. Jadidi, L. Gan, J.-K. Huang, J. W. Grizzle, and R. M. Eustice, "Hybrid contact preintegration for visual-inertial-contact state estimation using factor graphs," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 3783–3790.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [8] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask r-cnn," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2961–2969.
- [9] F. Dellaert, M. Kaess *et al.*, "Factor graphs for robot perception," *Foundations and Trends® in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017.
- [10] M. Kaess, H. Johannsson, R. Roberts, V. Ila, J. J. Leonard, and F. Dellaert, "isam2: Incremental smoothing and mapping using the bayes tree," *The International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, 2012.
- [11] J. Zhang and S. Singh, "Loam: Lidar odometry and mapping in real-time," in *Robotics: Science and Systems*, vol. 2, no. 9. Berkeley, CA, 2014, pp. 1–9.
- [12] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping," in *2020 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2020, pp. 5135–5142.